Specifica Funzionale — Nuova Funzionalità

Risk Evaluation

Functional Accelerator

Specifica per l'aggiunta della funzionalità di valutazione del rischio a partire da defect storici di progetto, con analisi AI-assisted e raccomandazioni preventive per nuovi deployment.

TIPO DOCUMENTO	VERSIONE	DESTINATARI
Specifica funzionale (TO-BE)	1.0 — Aprile 2026	Functional Analyst, Dev Team

SEZIONE 1

Contesto e motivazione

1.1 Problema

Nei progetti di delivery software dove un'applicazione viene estesa a nuovi mercati o paesi (rollout multi-country), i team di test si trovano spesso ad affrontare le stesse categorie di problemi già visti in deployment precedenti. Questa ripetizione è evitabile se si analizzano sistematicamente i difetti storici.

Il caso d'uso tipico: un team ha deployato KFC Austria e ha registrato decine di defect in UAT tramite un sistema ALM (es. HP ALM, Azure DevOps, Jira). Ora deve deployare KFC Slovakia con lo stesso stack. Senza una analisi preventiva, il team ripete gli stessi errori, con impatto su tempi e costi.

1.2 Soluzione proposta

Aggiungere al Functional Accelerator una funzionalità di **Risk Evaluation** che permette di:

- Caricare un export di defect da sistemi ALM (Excel o CSV)
- Ottenere una categorizzazione automatica dei defect per area funzionale e priorità
- Identificare i pattern e le root cause ricorrenti
- Ricevere raccomandazioni preventive specifiche per il nuovo deployment

> **Valore aggiunto:** il functional analyst ottiene in pochi minuti una "mappa del rischio" basata su evidenze reali, che può usare per pianificare i test, allocare risorse di QA e comunicare al cliente le aree critiche da presidiare.

1.3 Scope della specifica

In scope:

- Nuova sezione "Risk Evaluation" accessibile da ogni progetto
- Upload di file defect (Excel .xlsx/.xls, CSV) con colonne priorità e descrizione
- Analisi AI in due step: categorizzazione → raccomandazioni
- Visualizzazione risultati: grafici, tabella top defect, aree di rischio
- Storico delle valutazioni per progetto
- Feedback progressivo all'utente durante l'elaborazione

Out of scope (versione 1.0):

- Connessione diretta ad ALM (via API): l'utente esporta manualmente il file
- Grafici interattivi (chart.js, d3): visualizzazione come bar chart statiche
- Analisi su più di 200 defect (limite token Claude): file più grandi vengono troncati

- Export del report in PDF dall'interfaccia

SEZIONE 2

Casi d'uso

UC-01 — Avviare una Risk Evaluation

Attore	Functional Analyst / QA Manager
Precondizioni	Il progetto esiste nel sistema. L'utente ha un file export dall'ALM con i defect del progetto sorgente.
Trigger	L'utente accede alla sezione Risk Evaluation del progetto.
Flusso principale	1. L'utente carica il file defect (drag&drop o file picker) 2. Inserisce il nome del progetto sorgente (es. "KFC Austria") e del target (es. "KFC Slovakia") 3. Clicca "Run Risk Evaluation" 4. Il sistema avvia l'elaborazione asincrona e mostra il feedback di avanzamento 5. Al completamento, il sistema visualizza i risultati nella sezione destra
Postcondizioni	La valutazione è salvata nello storico del progetto e può essere richiamata in futuro.
Eccezioni	File vuoto o formato non riconosciuto → messaggio di errore specifico. Timeout Claude → errore con status "error" nella card storico.

UC-02 — Visualizzare i risultati di una valutazione precedente

Attore	Functional Analyst
Precondizioni	Esiste almeno una Risk Evaluation completata per il progetto.
Flusso principale	1. L'utente accede alla sezione Risk Evaluation 2. Dal pannello storico a sinistra, seleziona una valutazione precedente 3. Il pannello destro mostra i risultati: overall risk, grafici, top defect, raccomandazioni

UC-03 — Eliminare una valutazione

Attore	Functional Analyst
Flusso principale	L'utente clicca l'icona cestino sulla card storico → conferma → la valutazione viene eliminata dal DB.

UC-04 — Monitorare il progresso dell'elaborazione

Attore	Functional Analyst
Flusso principale	Durante l'elaborazione, il frontend fa polling ogni 2 secondi al backend. Il campo `progress_step` viene aggiornato ad ogni fase: "Parsing defect file..." → "Categorizing N defects..." → "Generating risk recommendations..." Il testo viene mostrato in real-time sotto la card della valutazione in corso.

SEZIONE 3

Requisiti funzionali

ID	Requisito	Priorità	Note
RF-01	Il sistema deve permettere l'upload di un file Excel (.xlsx, .xls) o CSV contenente i defect storici	Alta	Validazione MIME type lato backend; max 50MB
RF-02	L'utente deve poter specificare un "contesto sorgente" (nome progetto/country da cui provengono i defect) e un "contesto target" (deployment da analizzare)	Alta	Entrambi opzionali; se non specificati, Claude usa il nome del progetto come fallback
RF-03	L'elaborazione deve avvenire in modo asincrono con feedback progressivo in 3 fasi: parsing, categorizzazione, raccomandazioni	Alta	Il campo `progress_step` viene aggiornato ad ogni fase nel DB
RF-04	Il risultato deve includere: categorizzazione per area funzionale, distribuzione per priorità, top defect ricorrenti, pattern/root cause, aree di rischio con raccomandazioni	Alta	Struttura dati RiskAssessmentResult definita in types/index.ts
RF-05	Il livello di rischio complessivo deve essere classificato come: high / medium / low, con badge visivo prominente	Alta	Determinato da Claude sulla base dei dati aggregati
RF-06	I risultati devono essere visualizzati come bar chart orizzontali per categorie e distribuzione priorità	Media	Implementati come barre CSS proporzionali, senza librerie esterne
RF-07	Le valutazioni devono essere persistite nel DB e accessibili dallo storico del progetto	Alta	Tabella `risk_assessments`, FK su projects con ON DELETE CASCADE
RF-08	Ogni valutazione deve mostrare il numero di defect analizzati	Media	Campo `defect_count` nel DB; visibile nella card storico
RF-09	Il file caricato per l'analisi non deve essere archiviato permanentemente; deve essere eliminato dopo l'uso	Alta	File in `./tmp-uploads/`, cancellato in blocco `finally`
RF-10	In caso di errore durante l'elaborazione, il sistema deve aggiornare lo status a "error" e salvare il messaggio di errore nel DB	Alta	Visibile nella card storico con il testo dell'errore

SEZIONE 4

Requisiti non funzionali

ID	Requisito	Valore target
RNF-01	Tempo massimo di risposta per l'avvio (202 Accepted)	< 2 secondi
RNF-02	Tempo massimo di elaborazione completa (parsing + 2 chiamate Claude)	< 4 minuti (timeout configurato)
RNF-03	Dimensione massima file defect accettata	50 MB
RNF-04	Numero massimo di defect inviati a Claude per elaborazione	200 righe (cap per token limit)
RNF-05	Polling frontend durante elaborazione	ogni 2 secondi
RNF-06	Persistenza risultati	Indeterminata (fino a cancellazione manuale)
RNF-07	Nessuna dipendenza frontend aggiuntiva	Tutto con CSS puro e componenti esistenti

4.1 Note su scalabilità

Il limite di 200 defect è dovuto ai token disponibili nel contesto di Claude (`max_tokens: 4096` per risposta, ~30.000 chars di input). Per dataset più grandi, la versione futura dovrà implementare chunking (suddividere i defect in batch e aggregare i risultati) oppure RAG (embedding + retrieval dei defect più rilevanti).

SEZIONE 5

Architettura tecnica

5.1 Componenti nuovi

BACKEND — NUOVI

- `src/routes/risk.ts` — router Express con tutti gli endpoint
- Tabella SQLite `risk_assessments`
- Funzione `callClaudeJson<T>` in `claude.ts` — chiamata Claude generica con parse JSON tipizzato
- Funzioni locali in `risk.ts`: `parseCsvToRows`, `buildCategorizationPrompt`, `buildRecommendationPrompt`

FRONTEND — NUOVI

- `src/pages/RiskAssessmentPage.tsx` — pagina completa con pannello storico + risultati
- `riskApi` in `src/services/api.ts` — client API
- Tipi `RiskAssessment` e `RiskAssessmentResult` in `types/index.ts`
- Route `/projects/:id/risk-assessment` in `App.tsx`

5.2 Schema DB — tabella `risk_assessments`

Colonna	Tipo	Descrizione
id	TEXT PK	UUID v4
project_id	TEXT FK	→ projects(id) ON DELETE CASCADE
version_name	TEXT	Auto-generato: "Risk v1", "Risk v2"...
status	TEXT	pending | running | done | error
defect_count	INTEGER NULL	N. righe nel file caricato
result_json	TEXT NULL	JSON serializzato di RiskAssessmentResult
error_message	TEXT NULL	Messaggio se status = error
progress_step	TEXT NULL	Label step corrente durante l'elaborazione
created_at	TEXT	ISO datetime UTC

5.3 Schema JSON — RiskAssessmentResult

Campo	Tipo	Descrizione
summary	string	3-4 frasi sul profilo di rischio per il target
defectCategories[]	array	{ name, count, percentage } — gruppi per area funzionale, max 10

Campo	Tipo	Descrizione
priorityDistribution[]	array	{ priority, count, percentage } — distribuzione per priorità/severità
topDefects[]	array	{ title, count, priority, category } — i 5-8 issue più impattanti
patterns[]	string[]	4-6 pattern/root cause identificati
riskAreas[]	array	{ area, riskLevel, rationale, recommendation } — aree di rischio con azione preventiva
overallRiskLevel	'high'|'medium'|'low'	Livello di rischio complessivo

5.4 API endpoints

GET `/api/risk/:projectId`
Lista valutazioni del progetto, ordinate per data DESC

POST `/api/risk/:projectId/run`
Avvia valutazione — multipart/form-data: { file, sourceContext?, targetContext? } — risponde 202 Accepted

GET `/api/risk/:projectId/:assessmentId`
Dettaglio singola valutazione — include progress_step se in running

DELETE `/api/risk/:projectId/:assessmentId`
Elimina valutazione

SEZIONE 6

Flusso di elaborazione dettagliato

6.1 Step 1 — Parsing del file

Il backend legge il file dal filesystem temporaneo e lo converte in un array di oggetti JSON:

- **Excel (.xlsx, .xls):** tramite libreria `xlsx`, legge il primo sheet e usa `sheet_to_json()`
- **CSV:** parser interno che splitta per newline e virgola, gestisce valori quotati

Il numero di righe viene salvato in `defect_count`. Se il file è vuoto, viene lanciato un errore specifico.

6.2 Step 2 — Chiamata Claude: Categorizzazione

Il prompt inviato a Claude include:

- Il JSON dei primi 200 defect (cap 30.000 chars)
- Il contesto sorgente e il numero totale di defect
- Istruzione a produrre: `defectCategories`, `priorityDistribution`, `topDefects`, `patterns`

Claude legge le colonne presenti nel file (qualunque sia il nome — priorità, severity, description, title, summary...) e le interpreta autonomamente.

6.3 Step 3 — Chiamata Claude: Raccomandazioni

Il secondo prompt include:

- Il risultato JSON della categorizzazione (step 2)
- I contesti sorgente e target
- Istruzione a produrre: `summary`, `riskAreas[]`, `overallRiskLevel`

Claude usa la categorizzazione come evidenza per giustificare le raccomandazioni specifiche per il deployment target.

6.4 Aggregazione e salvataggio

I risultati di step 2 e 3 vengono uniti in un unico oggetto `RiskAssessmentResult` e salvati in `result_json`. Lo status passa a `done` e `progress_step` viene impostato a NULL.

1. **POST /run** — 202 Accepted, record creato con status=running, progress="Parsing defect file..."
2. **Parsing** — Excel/CSV letto, defect_count aggiornato, progress="Categorizing N defects..."
3. **Claude call #1** — categorizzazione, progress="Generating risk recommendations..."
4. **Claude call #2** — raccomandazioni, result_json aggiornato, status=done, progress=NULL

5 **Frontend polling** — rileva status=done, seleziona la valutazione, renderizza i risultati

SEZIONE 7

Interfaccia utente

7.1 Struttura della pagina

La pagina Risk Evaluation segue lo stesso layout del resto dell'applicazione: pannello sinistro fisso + pannello destro scorrevole.

PANNELLO SINISTRO

- Form upload: dropzone + campi sorgente/target + pulsante "Run Risk Evaluation"
- Storico valutazioni: card cliccabili con status badge, data, n. defect e progress step

PANNELLO DESTRO

- Stato vuoto se nessuna valutazione selezionata
- Spinner con progress step se in elaborazione
- Risultati completi se status = done
- Messaggio di errore se status = error

7.2 Componenti di visualizzazione risultati

Componente	Dati visualizzati	Design
Overall Risk Card	overallRiskLevel + summary	Icona shield colorata (rosso/ambra/verde) + testo narrativo
Defect Categories Chart	defectCategories[]	Bar chart orizzontale CSS con count e percentuale. Barre in purple-deep.
Priority Distribution Chart	priorityDistribution[]	Bar chart orizzontale CSS con colori per priorità (rosso=critico, ambra=medio, giallo=basso)
Top Recurring Issues	topDefects[]	Tabella: issue | categoria | priorità (dot colorato) | count
Patterns	patterns[]	Lista puntata con bullet in purple-deep
Risk Areas	riskAreas[]	Cards espanse: area + RiskBadge + rationale + azione preventiva (label "Action" in verde)

7.3 Navigazione

La Risk Evaluation è accessibile tramite un pulsante dedicato nell'header della pagina di dettaglio progetto. La route è `/projects/:id/risk-assessment`. Il pulsante Back in cima alla pagina riporta al dettaglio progetto.

7.4 Formati file accettati

- Excel: `.xlsx`, `.xls` — MIME: `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`, `application/vnd.ms-excel`
- CSV: `.csv` — MIME: `text/csv`, `text/plain`, `application/csv`

Formato consigliato del file defect: il file deve contenere almeno una colonna per la **priorità o severità** del defect e una colonna per la **descrizione** o **titolo**. Claude è in grado di interpretare qualsiasi naming delle colonne (es. "Priority", "Severity", "P1/P2/P3", "Title", "Summary", "Description"). Un formato tipico da ALM include anche: ID, Reporter, Component, Status, Resolution.

SEZIONE 8

Impatti sul sistema esistente

8.1 Modifiche ai file esistenti

File	Tipo modifica	Descrizione
`backend/src/db/index.ts`	Modifica	Aggiunta tabella `risk_assessments` nello schema SQL inizializzato all'avvio
`backend/src/server.ts`	Modifica	Import e registrazione del nuovo router: `app.use('/api/risk', riskRouter)`
`backend/src/services/claude.ts`	Modifica	Aggiunta funzione `callClaudeJson<T>`: chiamata Claude generica con parse JSON tipizzato. Non modifica le funzioni esistenti.
`backend/src/types/index.ts`	Modifica	Aggiunta interfacce `RiskAssessment` e `RiskAssessmentResult`. Nessuna modifica alle interfacce esistenti.
`frontend/src/types/index.ts`	Modifica	Mirror delle stesse interfacce lato frontend
`frontend/src/services/api.ts`	Modifica	Aggiunta oggetto `riskApi` con i metodi: list, get, run, delete
`frontend/src/App.tsx`	Modifica	Aggiunta route `/projects/:id/risk-assessment` → RiskAssessmentPage
`frontend/src/pages/ProjectDetailPage.tsx`	Modifica	Aggiunta navigazione verso la pagina Risk Evaluation (pulsante nell'header)

8.2 Nuovi file

File	Tipo	Descrizione
`backend/src/routes/risk.ts`	Nuovo	Router Express completo con tutti gli endpoint e la logica di elaborazione asincrona
`frontend/src/pages/RiskAssessmentPage.tsx`	Nuovo	Pagina React completa: form upload, polling, storico, visualizzazione risultati

8.3 Impatti su funzionalità esistenti

Nessun impatto breaking: la funzionalità Risk Evaluation è completamente additiva. Non modifica né il flusso di analisi funzionale né la struttura delle tabelle esistenti (`projects`, `files`, `analyses`, `impact_prototypes`). La nuova tabella `risk_assessments` viene creata con `CREATE TABLE IF NOT EXISTS`, quindi non causa errori su DB già esistenti.

8.4 Dipendenze

Nessuna nuova dipendenza npm. Le librerie usate (`xlsx`, `@anthropic-ai/sdk`) sono già presenti nel package.json del backend.

SEZIONE 9

Criteri di accettazione

ID	Criterio	Come verificarlo
CA-01	L'utente può caricare un file Excel con defect e avviare una valutazione	Caricare un file .xlsx con almeno 5 righe e colonne Priority + Description. Verificare che il backend risponda 202 e crei un record in `risk_assessments`.
CA-02	Il progresso viene aggiornato in tempo reale	Durante l'elaborazione, verificare che la card nel pannello storico mostri il testo dello step corrente che cambia nel tempo.
CA-03	Il risultato mostra l'overall risk level	A completamento, verificare che la sezione destra mostri un badge high/medium/low con icona shield appropriata.
CA-04	I grafici delle categorie mostrano dati reali	Verificare che le barre siano proporzionali ai valori di percentage restituiti da Claude e che i conteggi siano coerenti con il totale dei defect.
CA-05	Le aree di rischio hanno una raccomandazione specifica per il target	Ogni card in "Risk Areas" deve menzionare il contesto target specificato dall'utente.
CA-06	La valutazione persiste dopo reload della pagina	Dopo un refresh del browser, la valutazione completata deve essere ancora selezionabile dallo storico.
CA-07	Il file defect non viene salvato permanentemente	Verificare che dopo il completamento non esista nessun file in `./tmp-uploads/`.
CA-08	Un file non supportato viene rifiutato con messaggio di errore	Tentare l'upload di un PDF o JPG. Il backend deve rispondere 400 con messaggio "Only Excel (.xlsx, .xls) or CSV files are accepted".
CA-09	Eliminare una valutazione la rimuove dallo storico	Cliccare il cestino su una valutazione completata → confermare → verificare che non appaia più nella lista.
CA-10	La funzionalità non impatta l'analisi funzionale esistente	Eseguire un'analisi funzionale su un progetto esistente e verificare che i risultati siano identici a prima dell'aggiunta della feature.